

Chowly — Build Guide

Phases 2–5: Database, Backend Logic, UI, and Deployment

Your Senior Engineer Mentor

Contents

Before You Begin	2
Phase 2 — Database Schema & Migration	3
2.1 What we're building	3
2.2 Write the schema	3
2.3 Create the tables in your database	4
2.4 Seed some starting data	4
2.5 Checklist before Phase 3	5
Phase 3 — Server Actions (Backend Logic)	6
3.1 What are Server Actions?	6
3.2 Create a reusable database connection	6
3.3 Write the Server Actions	6
3.4 Checklist before Phase 4	8
Phase 4 — Building the UI	9
4.1 Structure	9
4.2 The homepage (loads data from the database)	9
4.3 The role toggle + header	9
4.4 The Customer screen	10
4.5 The Waiter screen	16
4.6 Run it and test the full flow	19
Phase 5 — Deploy to Vercel	20
5.1 Push your code to GitHub	20
5.2 Make sure the database client generates automatically on deploy	20
5.3 Import the project into Vercel	20
5.4 Get your live URL	21
5.5 Making future changes	21
5.6 You're done — final checklist	21

Before You Begin

This document continues directly from **Phase 1** (project setup, Next.js, Prisma, Neon database connection). It assumes you have a working `chowly` folder with `.env` pointing to a live Neon PostgreSQL database.

We'll go in order:

- **Phase 2** — Define the database tables (Prisma schema) and create them
- **Phase 3** — Write the backend logic (Server Actions)
- **Phase 4** — Build the Customer and Waiter screens
- **Phase 5** — Deploy to Vercel and get your public URL

Every code block tells you **exactly which file** it belongs to. Create files in VS Code by right-clicking the relevant folder → **New File**, and paste the path shown.

Phase 2 — Database Schema & Migration

2.1 What we're building

Three tables:

- **Menu** — the food/drink items customers can order
- **Staff** — the chefs and bartenders a waiter can assign
- **Order** — each customer order, its status, assigned staff, waiting time, and any complaint/rating

2.2 Write the schema

Open `prisma/schema.prisma` and **replace the entire file** with this:

```
// prisma/schema.prisma

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum StaffRole {
  chef
  bartender
}

enum OrderStatus {
  pending
  served
  paid
}

model Menu {
  id          String   @id @default(cuid())
  name        String
  price       Float
  preparationTime Int // minutes
  createdAt   DateTime @default(now())
}

model Staff {
  id   String   @id @default(cuid())
  name String
  role StaffRole
}

model Order {
  id          String   @id @default(cuid())
  items       Json      // array of { menuId, name, price, quantity, preparationTime }
  status      OrderStatus @default(pending)
  assignedChef String?
```

```

assignedBartender String?
waitingTime      Int // minutes, calculated when order is placed
complaint        String?
rating           Int? // 1-5
isPaid           Boolean @default(false)
createdAt        DateTime @default(now())
}

```

What each part means, briefly:

- `enum` — a fixed list of allowed values (a `StaffRole` can only ever be `chef` or `bartender`)
- `@id @default(cuid())` — every row gets a unique, auto-generated ID string
- `Json` — lets us store the list of ordered items (name, price, quantity) as one flexible field instead of a separate table
- `?` after a type (e.g. `String?`) — means the field is optional

2.3 Create the tables in your database

Run this command in your terminal (inside the `chowly` folder):

```
npx prisma migrate dev --name init
```

This does two things: 1. Creates the actual tables in your Neon database, matching the schema above 2. Generates a local `Prisma Client` — the TypeScript code you'll use to query the database

You should see output ending in something like `Your database is now in sync with your schema.`

If you get an error — double-check `.env` still has your correct Neon `DATABASE_URL`, and that there are no typos in the schema file.

2.4 Seed some starting data

Your app needs a starting `Menu` and `Staff` list to be usable — we'll write a "seed script" that inserts sample data once.

Install a small tool to run TypeScript scripts directly:

```
npm install -D tsx
```

Create a new file:

File: `prisma/seed.ts`

```

import { PrismaClient } from "@prisma/client";

const prisma = new PrismaClient();

async function main() {
  await prisma.staff.createMany({
    data: [
      { name: "Ada Okafor", role: "chef" },
      { name: "Femi Adeyemi", role: "chef" },
      { name: "Chidi Umeh", role: "chef" },
      { name: "Bola Johnson", role: "bartender" },
      { name: "Ngozi Eze", role: "bartender" },
      { name: "Tunde Bakare", role: "bartender" },
    ],
  });
}

```

```
});

await prisma.menu.createMany({
  data: [
    { name: "Jollof Rice & Chicken", price: 3500, preparationTime: 20 },
    { name: "Suya Platter", price: 4000, preparationTime: 15 },
    { name: "Pounded Yam & Egusi Soup", price: 3800, preparationTime: 25 },
    { name: "Grilled Fish", price: 5000, preparationTime: 30 },
    { name: "Chapman", price: 1500, preparationTime: 5 },
    { name: "Zobo Drink", price: 1000, preparationTime: 3 },
    { name: "Chilled Beer", price: 1200, preparationTime: 2 },
    { name: "Fresh Orange Juice", price: 1300, preparationTime: 5 },
  ],
});

console.log("Seed data inserted.");
}

main()
  .then(async () => {
    await prisma.$disconnect();
  })
  .catch(async (e) => {
    console.error(e);
    await prisma.$disconnect();
    process.exit(1);
  });
```

Now tell Prisma how to run this file. Open `package.json`, and add this **new top-level block** (as a sibling of "scripts", "dependencies", etc — not nested inside them):

```
"prisma": {
  "seed": "tsx prisma/seed.ts"
}
```

Run the seed:

```
npx prisma db seed
```

You should see `Seed data inserted.` printed.

Verify it worked (optional but recommended)

Prisma has a built-in visual database browser:

```
npx prisma studio
```

This opens a browser tab where you can see your Menu and Staff tables with the data you just inserted. Close the tab and press `Ctrl + C` in the terminal when you're done looking.

2.5 Checklist before Phase 3

- ☒ `prisma/schema.prisma` has Menu, Staff, and Order models
- ☒ `npx prisma migrate dev --name init` ran successfully
- ☒ `npx prisma db seed` inserted sample staff and menu items
- ☒ You can see the data in `npx prisma studio`

Phase 3 — Server Actions (Backend Logic)

3.1 What are Server Actions?

Normally, a web app needs a separate backend API. Next.js lets us skip that: a **Server Action** is a regular TypeScript function marked "use server" that runs only on the server, and can be called directly from your UI code as if it were a normal function — Next.js handles the network request behind the scenes. This is where all our database reads/writes will live.

3.2 Create a reusable database connection

Create this file:

File: `src/lib/prisma.ts`

```
import { PrismaClient } from "@prisma/client";

// Prevents creating a new database connection on every hot-reload during development
const globalForPrisma = globalThis as unknown as { prisma: PrismaClient };

export const prisma = globalForPrisma.prisma || new PrismaClient();

if (process.env.NODE_ENV !== "production") {
  globalForPrisma.prisma = prisma;
}
```

3.3 Write the Server Actions

Create this file:

File: `src/lib/actions.ts`

```
"use server";

import { prisma } from "@lib/prisma";
import { revalidatePath } from "next/cache";

type OrderItemInput = {
  menuId: string;
  name: string;
  price: number;
  quantity: number;
  preparationTime: number;
};

// ----- READ actions -----

export async function getMenu() {
  return prisma.menu.findMany({ orderBy: { name: "asc" } });
}

export async function getStaff() {
  return prisma.staff.findMany({ orderBy: { name: "asc" } });
}
```

```

export async function getAllOrders() {
  return prisma.order.findMany({ orderBy: { createdAt: "desc" } });
}

export async function getPendingOrders() {
  return prisma.order.findMany({
    where: { status: "pending" },
    orderBy: { createdAt: "asc" },
  });
}

export async function getOrder(orderId: string) {
  return prisma.order.findUnique({ where: { id: orderId } });
}

// ----- WRITE actions -----

// Customer places an order
export async function placeOrder(items: OrderItemInput[]) {
  if (!items || items.length === 0) {
    throw new Error("Cannot place an empty order.");
  }

  // Waiting time = the longest single item prep time (kitchen works in parallel)
  const waitingTime = Math.max(...items.map((i) => i.preparationTime));

  const order = await prisma.order.create({
    data: {
      items,
      waitingTime,
      status: "pending",
    },
  });

  revalidatePath("/");
  return order;
}

// Waiter assigns a chef and/or bartender to an order
export async function assignOrder(
  orderId: string,
  chefName: string,
  bartenderName: string
) {
  const order = await prisma.order.update({
    where: { id: orderId },
    data: {
      assignedChef: chefName || null,
      assignedBartender: bartenderName || null,
    },
  });

  revalidatePath("/");
}

```

```

    return order;
  }

  // Waiter marks an order as served
  export async function markOrderServed(orderId: string) {
    const order = await prisma.order.update({
      where: { id: orderId },
      data: { status: "served" },
    });

    revalidatePath("/");
    return order;
  }

  // Customer submits a complaint + rating (1-5) on a delayed order
  export async function submitComplaint(
    orderId: string,
    complaint: string,
    rating: number
  ) {
    if (rating < 1 || rating > 5) {
      throw new Error("Rating must be between 1 and 5.");
    }

    const order = await prisma.order.update({
      where: { id: orderId },
      data: { complaint, rating },
    });

    revalidatePath("/");
    return order;
  }

  // Customer makes a PRETEND payment – no real money moves, ever.
  export async function processPayment(orderId: string) {
    const order = await prisma.order.update({
      where: { id: orderId },
      data: { isPaid: true, status: "paid" },
    });

    revalidatePath("/");
    return order;
  }
}

```

A note on `revalidatePath("/")` — this tells Next.js “the data behind the homepage has changed, refresh it.” Without this, the app could show stale data after an action runs.

3.4 Checklist before Phase 4

- ☒ `src/lib/prisma.ts` exists
- ☒ `src/lib/actions.ts` exists with all 8 functions above
- ☒ Project still runs with no red errors: `npm run dev`

Phase 4 — Building the UI

4.1 Structure

We'll build:

File	Purpose
src/app/page.tsx	Loads initial data, renders the app shell
src/components/AppShell.tsx	Header + Customer/Waiter toggle switch
src/components/ContentView.tsx	Menu browsing, cart, order tracking, complaint, pretend payment
src/components/WaiterView.tsx	Pending orders, assign staff, mark served

4.2 The homepage (loads data from the database)

Replace the entire contents of `src/app/page.tsx` with:

```
import { getMenu, getStaff, getAllOrders } from "@lib/actions";
import AppShell from "@components/AppShell";

export default async function Home() {
  const [menu, staff, orders] = await Promise.all([
    getMenu(),
    getStaff(),
    getAllOrders(),
  ]);

  return <AppShell menu={menu} staff={staff} initialOrders={orders} />;
}
```

This is a **Server Component** — it runs on the server, fetches fresh data straight from the database on every page load, and passes it down as props.

4.3 The role toggle + header

Create the `src/components/` folder, then create:

File: `src/components/AppShell.tsx`

```
"use client";

import { useState } from "react";
import ContentView from "../ContentView";
import WaiterView from "../WaiterView";

export default function AppShell({
  menu,
  staff,
  initialOrders,
}: {
  menu: any[];
  staff: any[];
  initialOrders: any[];
}) {
```

```

const [role, setRole] = useState<"customer" | "waiter">("customer");

return (
  <main className="min-h-screen bg-gray-50">
    <header className="bg-white border-b sticky top-0 z-10">
      <div className="max-w-5xl mx-auto px-4 py-4 flex items-center justify-between">
        <h1 className="text-2xl font-bold text-orange-600">Chowly</h1>

        <div className="flex items-center gap-1 bg-gray-100 rounded-full p-1">
          <button
            onClick={() => setRole("customer")}
            className={`px-4 py-2 rounded-full text-sm font-medium transition ${
              role === "customer"
                ? "bg-orange-600 text-white"
                : "text-gray-600"
            }`}
          >
            Customer
          </button>
          <button
            onClick={() => setRole("waiter")}
            className={`px-4 py-2 rounded-full text-sm font-medium transition ${
              role === "waiter" ? "bg-orange-600 text-white" : "text-gray-600"
            }`}
          >
            Waiter
          </button>
        </div>
      </div>
    </header>

    <div className="max-w-5xl mx-auto px-4 py-6">
      {role === "customer" ? (
        <CustomerView menu={menu} />
      ) : (
        <WaiterView staff={staff} initialOrders={initialOrders} />
      )}
    </div>
  </main>
);
}

```

"use client" at the top means this component runs in the browser (needed because it uses useState for the toggle switch).

4.4 The Customer screen

File: `src/components/CustomerView.tsx`

```

"use client";

import { useEffect, useState } from "react";
import {
  placeOrder,

```

```

    getOrder,
    submitComplaint,
    processPayment,
  } from "@lib/actions";

type MenuItem = {
  id: string;
  name: string;
  price: number;
  preparationTime: number;
};

export default function CustomerView({ menu }: { menu: MenuItem[] }) {
  const [cart, setCart] = useState<Record<string, number>>({});
  const [orderId, setOrderId] = useState<string | null>(null);
  const [order, setOrder] = useState<any>(null);
  const [complaintText, setComplaintText] = useState("");
  const [rating, setRating] = useState(0);
  const [busy, setBusy] = useState(false);

  // Restore an in-progress order after a page refresh
  useEffect(() => {
    const saved = localStorage.getItem("chowly_order_id");
    if (saved) setOrderId(saved);
  }, []);

  // Poll the order's live status every few seconds
  useEffect(() => {
    if (!orderId) return;
    const fetchOrder = async () => setOrder(await getOrder(orderId));
    fetchOrder();
    const interval = setInterval(fetchOrder, 4000);
    return () => clearInterval(interval);
  }, [orderId]);

  const updateQty = (id: string, delta: number) => {
    setCart((prev) => {
      const next = { ...prev };
      const updated = Math.max(0, (next[id] || 0) + delta);
      if (updated === 0) delete next[id];
      else next[id] = updated;
      return next;
    });
  };

  const cartLines = menu
    .filter((m) => cart[m.id] > 0)
    .map((m) => ({ ...m, quantity: cart[m.id] }));

  const total = cartLines.reduce((s, l) => s + l.price * l.quantity, 0);

  const handlePlaceOrder = async () => {
    if (cartLines.length === 0) return;

```

```

    setBusy(true);
    const items = cartLines.map((l) => ({
      menuId: l.id,
      name: l.name,
      price: l.price,
      quantity: l.quantity,
      preparationTime: l.preparationTime,
    }));
    const newOrder = await placeOrder(items);
    localStorage.setItem("chowly_order_id", newOrder.id);
    setOrderId(newOrder.id);
    setOrder(newOrder);
    setCart({});
    setBusy(false);
  };

  const handleComplaint = async () => {
    if (!orderId || rating === 0) return;
    setBusy(true);
    setOrder(await submitComplaint(orderId, complaintText, rating));
    setBusy(false);
  };

  const handlePayment = async () => {
    if (!orderId) return;
    setBusy(true);
    setOrder(await processPayment(orderId));
    setBusy(false);
  };

  const startNewOrder = () => {
    localStorage.removeItem("chowly_order_id");
    setOrderId(null);
    setOrder(null);
    setComplaintText("");
    setRating(0);
  };

  // ----- Active order screen -----
  if (orderId && order) {
    const elapsedMinutes = Math.floor(
      (Date.now() - new Date(order.createdAt).getTime()) / 60000
    );
    const isDelayed = elapsedMinutes > order.waitingTime;

    return (
      <div className="max-w-lg mx-auto bg-white rounded-xl shadow p-6 space-y-5">
        <h2 className="text-xl font-semibold">Your Order</h2>

        <div className="space-y-1 text-sm text-gray-600">
          {order.items.map((it: any, idx: number) => (
            <div key={idx} className="flex justify-between">
              <span>

```

```

        {it.quantity} × {it.name}
      </span>
      <span>₹{(it.price * it.quantity).toLocaleString()}</span>
    </div>
  )})
</div>

<div className="border-t pt-3 flex justify-between font-medium">
  <span>Status</span>
  <span className="capitalize">{order.status}</span>
</div>

<div className="flex justify-between text-sm">
  <span>Estimated waiting time</span>
  <span>{order.waitingTime} min</span>
</div>

{order.assignedChef && (
  <div className="text-sm text-gray-600">Chef: {order.assignedChef}</div>
)}
{order.assignedBartender && (
  <div className="text-sm text-gray-600">
    Bartender: {order.assignedBartender}
  </div>
)}

{isDelayed && order.status !== "paid" && (
  <div className="bg-red-50 text-red-700 text-sm rounded-lg p-3">
    This order is running past its estimated time. Feel free to leave
    a complaint and rating below.
  </div>
)}

{!order.complaint && order.status !== "pending" && (
  <div className="space-y-2 border-t pt-4">
    <p className="text-sm font-medium">Rate this order</p>
    <div className="flex gap-1">
      {[1, 2, 3, 4, 5].map((n) => (
        <button
          key={n}
          onClick={() => setRating(n)}
          className={`text-2xl ${
            n <= rating ? "text-yellow-500" : "text-gray-300"
          }}`
        >
          ★
        </button>
      )})
    </div>
    <textarea
      value={complaintText}
      onChange={(e) => setComplaintText(e.target.value)}
      placeholder="Optional: describe the issue"
    >

```

```

        className="w-full border rounded-lg p-2 text-sm"
        rows={3}
      />
      <button
        onClick={handleComplaint}
        disabled={rating === 0 || busy}
        className="w-full bg-gray-800 text-white rounded-lg py-2 text-sm disabled:opaci
      >
        Submit rating & complaint
      </button>
    </div>
  )}

  {order.complaint !== null && order.rating !== null && (
    <div className="bg-gray-50 rounded-lg p-3 text-sm">
      You rated this order {order.rating}/5.
    </div>
  )}

  <div className="border-t pt-4 flex justify-between font-semibold">
    <span>Total</span>
    <span>#{total.toLocaleString() || "0"}</span>
  </div>

  {!order.isPaid ? (
    <button
      onClick={handlePayment}
      disabled={busy}
      className="w-full bg-orange-600 text-white rounded-lg py-3 font-medium disabled:opaci
    >
      Pay Now (PRETEND PAYMENT – no real charge)
    </button>
  ) : (
    <div className="space-y-3">
      <div className="bg-green-50 text-green-700 rounded-lg p-3 text-sm text-center">
        Payment received (simulated). Thank you!
      </div>
      <button
        onClick={startNewOrder}
        className="w-full border rounded-lg py-2 text-sm"
      >
        Start a new order
      </button>
    </div>
  )}
</div>
);
}

// ----- Menu / cart screen -----
return (
  <div className="grid md:grid-cols-3 gap-6">
    <div className="md:col-span-2 space-y-3">

```

```

<h2 className="text-xl font-semibold mb-2">Menu</h2>
{menu.map((item) => (
  <div
    key={item.id}
    className="bg-white rounded-xl shadow p-4 flex items-center justify-between"
  >
    <div>
      <p className="font-medium">{item.name}</p>
      <p className="text-sm text-gray-500">
        ₺{item.price.toLocaleString()} · ~{item.preparationTime} min
      </p>
    </div>
    <div className="flex items-center gap-3">
      <button
        onClick={() => updateQty(item.id, -1)}
        className="w-8 h-8 rounded-full border"
      >
        -
      </button>
      <span className="w-4 text-center">{cart[item.id] || 0}</span>
      <button
        onClick={() => updateQty(item.id, 1)}
        className="w-8 h-8 rounded-full border"
      >
        +
      </button>
    </div>
  </div>
)
)}}
</div>

<div className="bg-white rounded-xl shadow p-4 h-fit sticky top-24 space-y-3">
  <h3 className="font-semibold">Your Cart</h3>
  {cartLines.length === 0 && (
    <p className="text-sm text-gray-400">No items yet</p>
  )}
  {cartLines.map((l) => (
    <div key={l.id} className="flex justify-between text-sm">
      <span>
        {l.quantity} × {l.name}
      </span>
      <span>₺{(l.price * l.quantity).toLocaleString()}</span>
    </div>
  )
)}}
  {cartLines.length > 0 && (
    <div className="border-t pt-2 flex justify-between font-medium">
      <span>Total</span>
      <span>₺{total.toLocaleString()}</span>
    </div>
    <button
      onClick={handlePlaceOrder}
      disabled={busy}
    >
  )
}

```

```

        className="w-full bg-orange-600 text-white rounded-lg py-2 mt-2 disabled:opacity-50"
      >
        Place Order
      </button>
    </>
  )}
</div>
</div>
);
}

```

How a customer “keeps” their order across a refresh: we save the order’s ID in the browser’s `localStorage` (a small storage box built into the browser). The actual order data always lives in the database — `localStorage` just remembers *which* order this browser was looking at.

4.5 The Waiter screen

File: `src/components/WaiterView.tsx`

```

"use client";

import { useEffect, useState } from "react";
import { getPendingOrders, assignOrder, markOrderServed } from "@lib/actions";

type Staff = { id: string; name: string; role: "chef" | "bartender" };

export default function WaiterView({
  staff,
  initialOrders,
}: {
  staff: Staff[];
  initialOrders: any[];
}) {
  const [orders, setOrders] = useState(
    initialOrders.filter((o) => o.status === "pending")
  );
  const [selections, setSelections] = useState<
    Record<string, { chef: string; bartender: string }>
>({});

  const chefs = staff.filter((s) => s.role === "chef");
  const bartenders = staff.filter((s) => s.role === "bartender");

  const refresh = async () => setOrders(await getPendingOrders());

  useEffect(() => {
    const interval = setInterval(refresh, 5000);
    return () => clearInterval(interval);
  }, []);

  const handleAssign = async (orderId: string) => {
    const sel = selections[orderId];
    if (!sel) return;
    await assignOrder(orderId, sel.chef, sel.bartender);
  };

```



```

    await refresh();
  };

const handleServed = async (orderId: string) => {
  await markOrderServed(orderId);
  await refresh();
};

return (
  <div className="space-y-4">
    <h2 className="text-xl font-semibold">Pending Orders</h2>
    {orders.length === 0 && (
      <p className="text-gray-400 text-sm">No pending orders right now.</p>
    )}

    {orders.map((order) => {
      const sel = selections[order.id] || {
        chef: order.assignedChef || "",
        bartender: order.assignedBartender || "",
      };
      const elapsedMinutes = Math.floor(
        (Date.now() - new Date(order.createdAt).getTime()) / 60000
      );
      const isDelayed = elapsedMinutes > order.waitingTime;

      return (
        <div key={order.id} className="bg-white rounded-xl shadow p-4 space-y-3">
          <div className="flex justify-between items-start">
            <div>
              <p className="font-medium text-sm text-gray-500">
                Order #{order.id.slice(-6)}
              </p>
              <div className="text-sm">
                {order.items.map((it: any, i: number) => (
                  <div key={i}>
                    {it.quantity} × {it.name}
                  </div>
                ))}
              </div>
            </div>
            <div className="text-right">
              <p className="text-sm">Est. wait: {order.waitingTime} min</p>
              {isDelayed && (
                <p className="text-xs text-red-600 font-medium">Delayed</p>
              )}
            </div>
          </div>

          <div className="grid grid-cols-2 gap-3">
            <select
              value={sel.chef}
              onChange={(e) =>
                setSelections((prev) => ({

```

```

        ...prev,
        [order.id]: { ...sel, chef: e.target.value },
      )))
    }
    className="border rounded-lg p-2 text-sm"
  >
    <option value="">Assign chef...</option>
    {chefs.map((c) => (
      <option key={c.id} value={c.name}>
        {c.name}
      </option>
    ))}
  </select>

  <select
    value={sel.bartender}
    onChange={(e) =>
      setSelections((prev) => ({
        ...prev,
        [order.id]: { ...sel, bartender: e.target.value },
      )))
  >
    <option value="">Assign bartender...</option>
    {bartenders.map((b) => (
      <option key={b.id} value={b.name}>
        {b.name}
      </option>
    ))}
  </select>
</div>

<div className="flex gap-2">
  <button
    onClick={() => handleAssign(order.id)}
    className="flex-1 bg-gray-800 text-white rounded-lg py-2 text-sm"
  >
    Save Assignment
  </button>
  <button
    onClick={() => handleServed(order.id)}
    className="flex-1 bg-orange-600 text-white rounded-lg py-2 text-sm"
  >
    Mark as Served
  </button>
</div>
</div>
);
}}
</div>
);
}

```

4.6 Run it and test the full flow

```
npm run dev
```

Open **http://localhost:3000** and walk through this checklist:

1. Toggle to **Customer** → add a few items to the cart → **Place Order**
2. You should land on the order tracking screen showing status **pending** and an estimated waiting time
3. Toggle to **Waiter** → you should see that same order in the pending list
4. Assign a chef and bartender → **Save Assignment**
5. Click **Mark as Served**
6. Toggle back to **Customer** — within a few seconds the status should update to **served**, and a star rating + complaint box should appear
7. Submit a rating
8. Click **Pay Now (PRETEND PAYMENT)** — status should become **paid**
9. **Refresh the whole browser page** — your order should still be there, exactly as you left it (this proves the database is really persisting data, not just React memory)

If all nine steps work, your app is fully functional locally.

Phase 5 — Deploy to Vercel

5.1 Push your code to GitHub

Vercel deploys directly from a GitHub repository.

1. Go to <https://github.com> and sign in (or create a free account).
2. Click the **+** icon top-right → **New repository**.
3. Name it `chowly`, leave it **Public** or **Private** (your choice), do **not** initialize with a README (you already have a project). Click **Create repository**.
4. GitHub will show you a page with commands. Back in your terminal, inside the `chowly` folder, run:

```
git init
git add .
git commit -m "Initial commit: Chowly full-stack app"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/chowly.git
git push -u origin main
```

Replace `YOUR-USERNAME` with your actual GitHub username. If prompted, log in through the browser window that opens.

Your `.env` file will **not** be uploaded — Next.js's default `.gitignore` already excludes it. This is correct and intentional; we'll set the database URL directly in Vercel instead.

5.2 Make sure the database client generates automatically on deploy

Open `package.json` and check the `"scripts"` section. Add a `postinstall` line so Vercel always regenerates the Prisma client during deployment:

```
"scripts": {
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
  "lint": "next lint",
  "postinstall": "prisma generate"
}
```

Commit and push this change:

```
git add .
git commit -m "Add postinstall script for Prisma"
git push
```

5.3 Import the project into Vercel

1. Go to <https://vercel.com> and sign up/log in — choose **Continue with GitHub** so it can see your repositories.
2. Click **Add New...** → **Project**.
3. Find `chowly` in the list and click **Import**.
4. Vercel will auto-detect it as a Next.js project — leave the build settings as default.
5. Before clicking **Deploy**, expand **Environment Variables** and add:

Name	Value
DATABASE_URL	<i>paste your Neon connection string, same one from your .env</i>

6. Click **Deploy**.

Vercel will install dependencies, run `prisma generate` (thanks to the postinstall script), build the app, and deploy it. This takes 1–3 minutes.

5.4 Get your live URL

When it finishes, Vercel shows a success screen with a **Visit** button and a URL like:

`https://chowly-yourname.vercel.app`

That's your **public, live URL** — anyone can open it. Click through the same 9-step test from Phase 4 on the live site to confirm everything works in production too.

Because your live app and your local app both point at the **same Neon database**, any order placed on the live URL will also show up if you run the app locally, and vice versa. That's expected for this project — it's one shared database.

5.5 Making future changes

Any time you edit code locally and want the live site updated:

```
git add .
git commit -m "describe what you changed"
git push
```

Vercel automatically redeploys within a minute or two of every push to `main`.

5.6 You're done — final checklist

- ☒ Code pushed to GitHub
- ☒ DATABASE_URL environment variable set in Vercel
- ☒ Deployment succeeded with a live `.vercel.app` URL
- ☒ Full customer → waiter → payment flow tested on the live URL
- ☒ Refreshing the live page preserves order data

Congratulations — Chowly is live. From here, natural next steps (only if you want them later) would be adding real authentication, a proper payment provider like Paystack or Stripe, and email/SMS notifications when an order status changes. None of that is required for what you set out to build.